Copyright All rights reserved

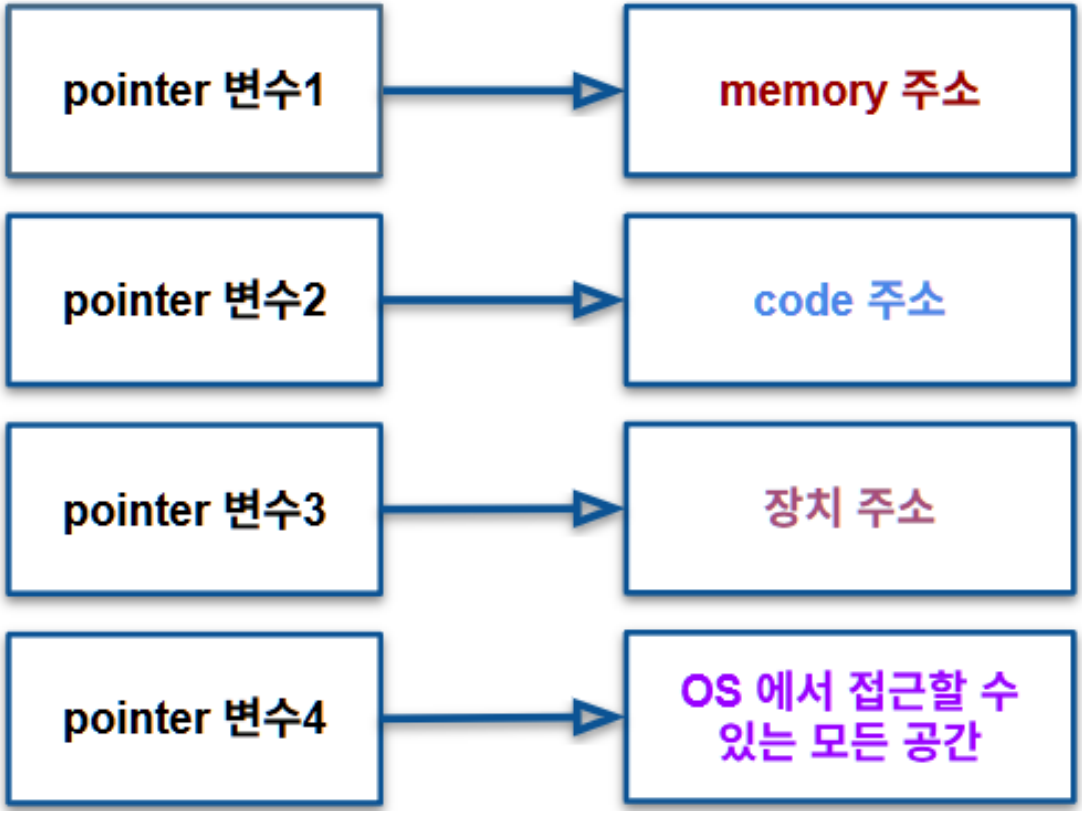
09. 포인터
(Pointer)

Heesung Oh
https://github.com/3dapi

9.1 주소와 참조, 포인터

Heesung Oh
Copyright All rights reserved

- **포인터 변수의 범위**
  - 시스템(OS)이 허용하는 주소 공간 내의 모든 위치 참조
- **Ex)**
  - 메모리 영역 (일반 변수)
  - 코드 영역 (함수)
  - 장치 영역 (레지스터)



9.1.1 포인터 선언과 의미

Heesung Oh
Copyright All rights reserved

- 포인터
  - 주소를 저장하는 변수
  - 주소 → 데이터 접근

- 포인터 선언

```
자료형* 포인터_이름;
```

```
// 같은 자료형을 가지고 선언하는 경우
자료형 *포인터_이름1, *포인터_이름2, *포인터_이름3;
```

9.1.1 포인터 선언과 의미

Heesung Oh
Copyright All rights reserved

- **포인터 선언 예**

```
// 일반 변수: 자료형 변수_이름 = 값

int myIntValue = 1000;



// 포인터: 자료형* 포인터변수_이름 = 값(유효한 주소)

int* ptr = &myIntValue;   // ptr은 myIntValue의 주소를 저장



// 포인터의 형변환: (자료형*)(유효한 주소)

char* ptr = &myIntValue;          // C: 경고, C++: 오류
char* ptr = (char*)&myIntValue;   // 주소를 char*로 강제 변환
```

9.1.1 포인터 선언과 의미

Heesung Oh
Copyright All rights reserved

- **포인터 선언 배열/리터럴**

```
자료형* 포인터_이름 = 배열_이름;

int arr_int[] = {1, 2, 3, 4};
int* ptr_int = arr_int;   // 배열 이름은 arr_int[0]의 주소


char arr_char[] = {'H', 'e', 'l', 'l', 'o'};
char* ptr_char = arr_char;   // arr_char[0]의 주소


char arr_str[] = {'H', 'e', 'l', 'l', 'o', '\0'};
char* ptr_str = arr_str;


const char* ptr_str = "Hello world";   // 읽기 전용 주소
```

9.1.2 포인터의 데이터 접근

Heesung Oh
Copyright All rights reserved

- **포인터의 데이터 접근 방법**
  - **간접 참조 연산자: * 연산자**
  - **첨자 연산자: [] 연산자**

p[0] ↔ *p

```
*포인터_이름 = 값;          // 간접 참조 연산으로 값 설정
포인터_이름[index] = 값;  // 첨자 연산으로 값 설정
```

```
int myIntValue = 0x11223344;
int* int_ptr = &myIntValue;

printf("변수의 주소: %p , 포인터에 저장된 주소: %p\n", &myIntValue, int_ptr);

printf("값 변경 전 (주소: %p) = %#x\n", int_ptr, myIntValue);
*int_ptr = 0x12345678;
printf("값 변경 후 (주소: %p) = %#x\n", int_ptr, myIntValue);

// 단일 변수 주소에 인덱스 사용은 권장하지 않음
int_ptr[0] = 0x87654321;   // 같은 의미로 동작하지만 메모리 구조를 이해해야 함
printf("다시 변경 후 (주소: %p) = %#x\n", int_ptr, myIntValue);
```

9.1.2 포인터의 데이터 접근

Heesung Oh
Copyright All rights reserved

- 변수의 주소 → &
- 배열의 주소 → &p
- 배열 요소의 시작 주소 → &p[0] or p

- 배열 요소의 값 → p[0]
- 포인터 주소에 저장된 값 → *p

&p[0] ↔ p ,

p[0] ↔ *p

9.1.2 포인터의 데이터 접근

Heesung Oh
Copyright All rights reserved

- 의도적으로 포인터를 캐스팅
  첨자 연산 메모리

```
// Little Endian 기준
int myColor = 0x11223344;
int*  int_ptr  = &myColor;
char* char_ptr = (char*)&myColor;   // 주소를 char*로 해석

*char_ptr = 0x44;   // 가장 낮은 1바이트 변경
printf("결과 (주소: %p) = %#x\n", int_ptr, myColor);

// RGBA 32bit 색상을 다룰 때 많이 사용하는 방식
char_ptr[0] = 0x99;
char_ptr[1] = 0x88;
char_ptr[2] = 0x77;
char_ptr[3] = 0x66;

printf("최종 결과 (주소: %p) = %#x\n", int_ptr, myColor);
```

9.1.3 엔디안 확인

Heesung Oh
Copyright All rights reserved

- Little Endian: 낮은 바이트 → 낮은 주소
- Big Endian: 높은 바이트 → 낮은 주소

```c
#include <stdio.h>

int main(void)
{
    unsigned int x = 0x12345678;
    unsigned char* p = (unsigned char*)&x;
    // 메모리에 실제로 저장된 바이트 순서를 확인한다.
    printf("%#x 저장 순서 => %#x %#x %#x %#x\n", x, p[0], p[1], p[2], p[3]);

    if (p[0] == 0x78)
        printf("현재 시스템은 Little Endian 입니다.\n");
    else if (p[0] == 0x12)
        printf("현재 시스템은 Big Endian 입니다.\n");
    else
        printf("알 수 없는 Endian 입니다.\n");

    return 0;
}
```

9.2 포인터 연산

Heesung Oh
Copyright All rights reserved

- 포인터 연산
  - 주소 기반 연산
  - 메모리 주소 기반
- 사용 가능한 연산
  - 대입 연산 (=)
  - 산술 연산 (+, –)
  - 비교 연산 (==, !=, 〈, 〉)
  - 첨자 연산 ([])

- 핵심 특징
  - 포인터는 단순한 정수가 아님
  - 타입 크기(sizeof)에 따라 연산 결과 결정됨
  - 모든 연산이 허용되지만, 모두 의미 있는 것은 아님

- 주의 사항
  - 의미 있는 경우에만 사용, 항상 유효한 주소인지 확인 필요
  - 잘못된 연산 → 메모리 오류 발생

9.2.1 포인터 산술 연산

Heesung Oh
Copyright All rights reserved

연산식	의미	실제 주소
ptr + n	n번째 요소 앞으로 이동	**ptr + n * sizeof(자료형)**
ptr - n	n번째 요소 뒤로 이동	**ptr - n * sizeof(자료형)**
ptr++	표현식의 평가가 끝난 뒤 다음 요소로 이동하는 후위 증가	**ptr + sizeof(자료형)**
++ptr	먼저 포인터를 이동시키고, 이동된 주소로 표현식을 평가하는 전위 증가	**ptr + sizeof(자료형)**
ptr--	표현식의 평가가 끝난 뒤 이전 요소로 이동하는 후위 감소	**ptr - sizeof(자료형)**
--ptr	먼저 포인터를 이동시키고, 이동된 주소로 표현식을 평가하는 전위 감소	**ptr - sizeof(자료형)**

9.2.1 포인터 산술 연산

Heesung Oh
Copyright All rights reserved

- 포인터 이동
  - 자료형 크기(sizeof) 기준으로 이동
  - int* + 1 → 4 바이트(int size) 이동
  - char* + 1 → 1 바이트(char size) 이동

- 전위 / 후위 증가
  - ++ptr : 먼저 이동 후 사용
  - ptr++ : 사용 후 이동

- 주의!!

```
value = *ptr++;

// 해석
*ptr++ == *(ptr++)
temp = *ptr;
ptr++;
value = temp;
```

9.2.1 포인터 산술 연산

Heesung Oh
Copyright All rights reserved

- 배열과 포인터
  - 배열 이름 → 0번째 요소의 주소로 변환
  - 배열은 포인터처럼 사용 가능

```
int arr[] = {...};

// 배열 표현식이 포인터로 암묵적으로 변환
// (arr → &arr[0])
int* ptr = arr;

// 명시적 표현, 위와 동일
int* ptr2 = &arr[0];
```

9.2.1 포인터 산술 연산

Heesung Oh
Copyright All rights reserved

- 첨자 연산의 본질
  - arr[i] == *(arr + i)
  - a[b] == *(a + b)

  - a[b] == b[a] (동일한 의미)
    - a[b] → (a + b) → (b + a) == b[a]
    - 실제 코드에서는 사용하지 않음 (가독성 문제)

```
6[arr]; // arr[6]와 동일
```

- 표현 방식 비교
  - arr[i] → 가독성 중심
  - *(ptr + i) → 포인터 연산 명확

9.2.1 포인터 산술 연산

Heesung Oh
Copyright All rights reserved

- 배열 vs 포인터
  - 배열: 고정된 메모리 공간
  - 포인터: 주소를 저장하는 변수

- 핵심
  - 배열 접근 = 포인터 산술 연산
  - 배열의 첨자 연산 == 포인터의 첨자 연산과 동일

```
int arr[10];
int* ptr = arr;

arr[i] == *(arr + i) // 배열
ptr[i] == *(ptr + i) // 포인터
```

9.2.1 포인터 산술 연산

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>
int main(void)
{
    int arr[9] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

    // arr[6]와 동일한 표현
    int value = 6[arr];

    printf("6[arr] = %d\n", 6[arr]);


    // 배열 표현식은 암시적으로 포인터로 변환. ptr = &arr[0]; 와 동일.
    int* ptr = arr;

    // 명시적 표현, ptr2 = arr; 와 동일
    int* ptr2 = &arr[0];

    for(int i=0; i<9; ++i)
    {
        printf("arr[%d] = %d, *(arr+%d) =%d,  *(ptr+%d) = %d\n"
            , i, i[arr], i, *(arr+i), i, *(ptr+i));
        // arr[i] 대신 i[arr]로 써도 되지만 비추천.
    }

    return 0;
}
```

9.2.2 포인터와 연산자 우선순위

Heesung Oh
Copyright All rights reserved

- 연산자 우선순위
  - 후위 증가 p++ → 가장 높음
  - 전위 증가 ++p, * → 동일
  - 결합 방향 → 오른쪽 → 왼쪽
- *p++
  - *p++ == *(p++)
  - 현재 주소 값 읽기 → 이후 포인터 증가
- (*p)++
  - 값 증가
  - 포인터 이동 없음
- *++p
  - *(++p)
  - 포인터 먼저 이동 → 값 읽기

9.2.2 포인터와 연산자 우선순위

Heesung Oh
Copyright All rights reserved

```
int arrInt[] = {1000, 2000, 3000, 4000};
int* ptr = arrInt;
printf("++*ptr++: %d\n", ++*ptr++);
printf("*ptr: %d\n", *ptr);
```

1. ptr++
   현재 주소 사용 → arrInt[0]
   이후 ptr → arrInt[1]

2. *(ptr++)
   증가 전 주소 값 참조
   → 1000

3. ++
   값 증가
   → 1001

4. 결과
   arrInt[0] → 1001로 변경 → 1001 출력
   ptr → arrInt[1] 가리킴 → 2000 출력

9.2.3 포인터 사이의 유효한 연산

Heesung Oh
Copyright All rights reserved

연산자	의미
ptr1 == ptr2	두 포인터가 같은 주소를 가리키는지 비교
ptr1 != ptr2	두 포인터가 다른 주소를 가리키는지 비교
ptr1 < ptr2	같은 배열 내부에서, 앞쪽 요소를 가리키는지 비교
ptr1 > ptr2	같은 배열 내부에서, 뒤쪽 요소를 가리키는지 비교
ptr1 <= ptr2	같은 배열 내부에서 상대적 위치 비교
ptr1 >= ptr2	같은 배열 내부에서 상대적 위치 비교
ptr ± n	정수형 n 값 만큼 요소 단위로 이동
ptr1 - ptr2	같은 배열 내부에서 두 포인터 사이의 요소 개수

9.2.3 포인터 사이의 유효한 연산

Heesung Oh
Copyright All rights reserved

```
int arr[] = {10, 20, 30, 40, 50, 60};

int* p = &arr[1];
int* q = &arr[5];

printf("p < q : %d\n"  , p < q);  //  1 (의미 있는 비교)
printf("p == q: %d\n"  , p == q); //  0
printf("q - p : %lld\n", q - p);  //  4 (요소 차이)
printf("p - q : %lld\n", p - q);  // -4
```

9.3.1 포인터와 배열의 차이

Heesung Oh
Copyright All rights reserved

- 기본 개념
  - 포인터 → 주소 를 저장하는 변수 (변경 가능)
  - 배열 → 데이터 를 저장하는 고정된 메모리 공간
- 핵심 차이
  - 포인터 → 다른 주소를 가리키도록 변경 가능
  - 배열 이름 → 변경 불가 (고정된 위치)
- sizeof 차이
  - sizeof(배열) → 전체 메모리 크기
  - sizeof(ptr) → 포인터 변수 크기, 타입과 무관
  - char*, int*, void*, struct* → 모두 동일
- 동일한 주소

```
ptr = arr    ;
ptr = &arr[0];
```

9.3.2 다차원 배열과 포인터 관계

Heesung Oh
Copyright All rights reserved

- 다차원 배열 → 메모리상 연속된 1차원 배열

- 접근 방식
  - 인덱스 방식
  - 포인터 산술 + 간접 연산자(*)
  - 다차원 배열 접근 = 중첩된 포인터 연산 (* + + 연산 반복)

```
arr[i][j][k][l]  ==

      *(*(*(*(arr + i) + j) + k) + l)
```

9.3.3 포인터의 배열

Heesung Oh
Copyright All rights reserved

- 구조
  - 포인터 배열 → 포인터를 요소로 가지는 배열
    - 포인터 배열 = 주소를 저장하는 배열
  - 각 요소 → 변수의 주소 저장

- 선언
  - 자료형* 포인터배열이름[배열크기] = { NULL };
  - 포인터는 NULL로 초기화하는 습관이 중요

- 초기화되지 않은 포인터 사용
  - → 유효하지 않은 주소 접근
  - → 의도하지 않은 메모리 접근

- 접근 방식
  - ptr[i] → 주소
  - *ptr[i] → 실제 값

9.3.3 포인터의 배열

Heesung Oh
Copyright All rights reserved

```
int val1 = 10;
int val2 = 20;
int val3 = 30;
int val4 = 40;

// 주소 저장
int* ptr[] = { &val1, &val2, &val3, &val4 };

//  문자열 배열.
const char *str_list [] =
        { "Hello", "World", "Hi", "Welcome" };
```

9.3.4 배열 포인터

Heesung Oh
Copyright All rights reserved

- **배열 포인터 (Array Pointer)**
  - 배열 전체를 가리키는 포인터
  - 배열의 크기 정보 포함 (타입에 포함됨)
- **타입 불일치 주의**
  - 포인터는 타입 기준으로 메모리를 해석
  - 타입이 다르면 읽는 단위가 달라짐
  - 잘못된 타입 → 위험한 접근

```
// 선언
자료형 (*p)[읽어들일_배열_요소_길이];

// 선언과 동시에 초기화
// 배열 크기의 불일치에 대한 정의되지 않은 동작 주의.
자료형 (*p)[읽어들일_배열_요소_길이] = &배열_이름;
```

```
int (*ptr_arr)[4] = &arr;
```

Heesung Oh
Copyright All rights reserved

- 크기 차이로 인한 타입 불일치의 UB 예

```
int (*ptr_arr)[4] = &arr;
```

```
ptr_arr[0] → [ 1  2  3  4 ]
ptr_arr[1] → [ 5  6  7  8 ]
ptr_arr[2] → [ 9 10 11 12 ]
ptr_arr[3] → [13 14 ?? ?? ]
// 배열 크기를 초과한 영역까지 포함해 해석
```

```
출력 예:
ptr_arr 1: [ 1  2  3  4]
ptr_arr 2: [ 5  6  7  8]
ptr_arr 3: [ 9 10 11 12]
ptr_arr 4: [13 14 -858993460 -858993460]
```

9.3.4 배열 포인터

Heesung Oh
Copyright All rights reserved

- 올바른 방법
  - 고정 크기 배열 구조 사용
  - typedef를 통한 타입 명확화
  - 타입과 실제 메모리 구조 일치

```
#define BLOCK 4
typedef int CHUNK_INT[BLOCK];
void printChunk(int rows, int cols, CHUNK_INT* pChunk);
```

```
int arr[][BLOCK] = {
        {10, 11, 12, 13},
        {14, 15, 16, 17},
        {17, 18, 19, 20}
    };
    int rows = sizeof(arr) / sizeof(arr[0]);  // 행의 수.
    printChunk(rows, BLOCK, arr);
```

9.4.1 문자열 리터럴과 포인터

Heesung Oh
Copyright All rights reserved

- 문자열 리터럴
  - 실행 시 수정 불가능
  - 읽기 전용 영역에 저장
  - 프로그램 전체에서 공유될 수 있음
  - 문자열을 수정하려면 배열로 선언

```
// C에서 허용, C++에서는 허용 안됨
// const char* 사용 권장
char* s = "Hello";  // 읽기 전용 메모리의 주소
```

```
const char* s = "Hello";
```

```
char s[] = "Hello";  // 리터럴을 배열 공간에 복사
```

9.4.2 sizeof 와 strlen 비교

Heesung Oh
Copyright All rights reserved

- sizeof vs strlen
  - sizeof → 메모리 크기
  - strlen → 문자열 길이

```
char* p = "Hello";
char s[] = "Hello";

sizeof(p);   // 포인터 p의 크기 (4 또는 8 바이트)
sizeof(s);   // 배열 전체 크기 (문자 5개 + '\0' → 6)
strlen(p);   // 문자열 길이 5
strlen(s);   // 문자열 길이 5
```

9.4.3 문자열 풀(string literal pool)

Heesung Oh
Copyright All rights reserved

- **문자열 풀 (String Literal Pool)**
  - 문자열 리터럴은 컴파일 시 모여서 저장됨
  - 읽기 전용 영역에 저장
  - 동일한 문자열은 메모리를 공유할 수 있음
    → 항상 같은 주소를 보장하지 않음

```
const char* a = "Hi";
const char* b = "Hi";
```

구분	문자열 리터럴	배열
저장	읽기 전용	일반 메모리
수정	불가능	가능
공유	가능 (컴파일러에 따라 다름)	불가능

9.4.4 문자열과 포인터 실습 예제

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
// 1) 문자열 리터럴과 배열의 차이
// C에서는 허용되지만, C++에서는 허용되지 않는다.
// 문자열 리터럴은 수정할 수 없으므로 const char* 사용 권장

    char* p = "World";      // 읽기 전용 메모리
    char  s[] = "World";    // 배열 공간에 복사

    printf("초기 상태\n");
    printf("p  문자열: %s\n", p);
    printf("s  문자열: %s\n\n", s);

    // 문자열 리터럴 수정 시도 (위험 코드, 주석을 풀고 테스트해 볼 것)
    // p[0] = 'A';   // 정의되지 않은 동작(Undefined Behavior)

    // 배열은 수정 가능
    s[0] = 'A';
    printf("배열 s 수정 후 문자열: %s\n\n", s);

    // 2) sizeof vs strlen 차이 확인
    printf("sizeof(p) = %zu (포인터 크기)\n", sizeof(p));
    printf("sizeof(s) = %zu (배열 전체 크기)\n", sizeof(s));
    printf("strlen(p) = %zu (문자열 길이)\n", strlen(p));
    printf("strlen(s) = %zu (문자열 길이)\n\n", strlen(s));
```

```c
    // 3) 문자열 복사 후 뒤집기
    char src[] = "Hello";
    char buf[32]={0};

    strcpy(buf, src);   // 문자열 복사: 복사 대상인 buf 크기가
충분해야 함
    int len = (int)strlen(buf);

    printf("원본 문자열: %s\n", buf);
    printf("역순 출력   : ");
    for(int i = len - 1; i >= 0; --i)
    {
        printf("%c", buf[i]);
    }

    printf("\n\n");

    // 4) 문자열 리터럴 풀 확인 (주소 비교)
    const char* a = "Hi";
    const char* b = "Hi";

    printf("문자열 리터럴 풀 테스트\n");
    printf("a 주소: %p\n", a);
    printf("b 주소: %p\n", b);
    if (a == b)
        printf("a와 b는 같은 문자열 리터럴을 가리킨다.\n");
    else
        printf("a와 b는 다른 위치를 가리킨다.\n");
```

9.5 문자열 함수

Heesung Oh
Copyright All rights reserved

- 헤더파일
  - 〈string.h〉
- 널 문자('\0') 기준으로 동작

분류	함수
길이	strlen
복사	strcpy, strncpy, memcpy, memmove
비교	strcmp, strncmp, stricmp / strcasecmp
결합	strcat, strncat
문자 검색	strchr, strrchr
문자열 검색	strstr
토큰 분리	strtok
문자열 → 숫자	atoi, atof, strtol, strtoul, strtod
길이 분석	strspn, strcspn

9.5.1 strlen – 문자열 길이

Heesung Oh
Copyright All rights reserved

- 동작
  - 탐색: 시작 주소 → NULL문자('\0')까지 순차 스캔
  - 반환: '\0' 제외, 순수 문자 개수 O(n)

- 주의
  - 데이터 타입: size_t (부호 없는 정수)
  - 메모리: sizeof(배열 크기) ≠ strlen(문자열 길이)
  - 위험: NULL 문자 미 존재 시 메모리 침범(Overrun)

9.5.2 strcpy / strncpy – 문자열 복사

Heesung Oh
Copyright All rights reserved

- 동작
  - strcpy:전체 복사 ('\0' 포함)
  - strncpy: 최대 n바이트 복사
    - src 길이 < n → 남은 공간 '\0' 채움
    - src 길이 ≥ n → '\0' 없음
- 주의
  - strcpy: 이 검사 없음 → 버퍼 오버플로
  - strncpy:문자열 종료 보장 안됨
  - '\0' 없으면 문자열 함수 사용 불가
- 안전 패턴

```
// 모든 요소 '\0' 으로 초기화
char str_buf[8] = {0};

// 마지막 한 칸을 널 문자를 위해 비워 둔다
strncpy(str_buf, src, sizeof(str_buf)-1);
```

9.5.3 strcmp / strncmp – 문자열 비교

Heesung Oh
Copyright All rights reserved

- 동작
  - strcmp
    - 앞에서부터 비교 → 첫 차이에서 결정
    - 결과: 0 / 음수 / 양수
  - strncmp
    - 앞에서 n글자까지만 비교
    - → n 이후 비교안함

- 주의
  - 문자값 기준 비교 (ASCII) → 대소문자 구분
  - 문자열은 반드시 '\0' 필요

```
int r0= strcmp("abc", "abc"); // 0 (같음)
int r1= strcmp("abc", "abd"); // 음수 (왼쪽이 앞)
int r2= strcmp("dog", "cat"); // 양수 (왼쪽이 뒤)
```

9.5.3 stricmp / strcasecmp – 문자열 비교

Heesung Oh
Copyright All rights reserved

- 비표준 대소문자 구분 없이 비교

- Windows
  - stricmp, strnicmp
  - _stricmp, _strnicmp (MSVC 권장)

- POSIX (Linux, macOS)
  - strcasecmp, strncasecmp

9.5.4 strcat / strncat – 문자열 이어 붙이기

Heesung Oh
Copyright All rights reserved

- 동작
  - strcat
    - '\0' 위치 찾은 뒤 이어 붙임
  - strncat
    - 최대 n글자만 추가
    - 마지막에 '\0' 추가
- 주의
  - strcat: 길이 검사 없음
    → 버퍼 오버플로우
  - strncat: n은 "추가 길이"
    (전체 크기 아님)
  - 남은 공간 직접 계산 필요

```
char buf[32] = "Hello";
strcat(buf, " World");
// "Hello World"
```

9.5.5 strchr – 특정 문자 검색

Heesung Oh
Copyright All rights reserved

- 동작
  - 문자열에서 특정 문자 검색
  - 처음 발견된 위치의 주소 반환
- 주의
  - 찾지 못하면 → NULL 반환
  - 반환 값은 문자열의 일부를 가리키는 포인터
    → 이후 문자열처럼 사용 가능
  - '\0' 도 검색 가능
    → 문자열 끝 위치 반환

```
char text[] = "Hello World";
char* p = strchr(text, 'o');
// p 출력: o World
```

9.5.6 strstr – 부분 문자열 검색

Heesung Oh
Copyright All rights reserved

- 동작
  - 문자열에서 특정 문자 검색
  - 처음 발견된 위치의 주소 반환
- 주의
  - 찾지 못하면 → NULL 반환
  - 반환값은 문자열의 일부를 가리키는 포인터
    → 이후 문자열처럼 사용 가능
  - '\0' 도 검색 가능
    → 문자열 끝 위치 반환

```
char text[] = "Game Development in C";
char* p = strstr(text, "Develop");

// p 출력: Development in C
```

9.5.7 strtok – 문자열 분리(기본)

Heesung Oh
Copyright All rights reserved

- 동작
  - 구분자(delimiter) 기준으로 문자열 분리
  - 첫 호출 → 첫 번째 토큰 반환
  - 이후 → strtok(NULL, ...)로 계속 추출

- 주의
  - 원본 문자열 변경됨 → 구분자 위치에 `'\0'` 삽입
  - 문자열 리터럴 사용 불가 → 반드시 수정 가능한 배열 사용
  - 내부 상태 사용
    → 동시에 여러 문자열 처리 불가 / thread unsafe

9.5.7 strtok – 문자열 분리(기본)

Heesung Oh
Copyright All rights reserved

```
char s[] = "C,C++,Java,Lua";
char* token = strtok(s, ",");

while (token != NULL)
{
    printf("%s\n", token);
    token = strtok(NULL, ",");
}
```

```
호출 전:  "C,C++,Java,Lua\0"
호출 후:  "C\0C++\0Java\0Lua\0"
            ↑    ↑     ↑     ↑
          token1 token2 ...
```

9.5.8 문자열 함수 통합 실습 예제

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    printf("=== 1. strlen ===\n");
    char* str = "Hello";
    size_t len= strlen(str);
    printf("문자열: %s ==> length: %zu\n", str, len);

    printf("=== 2. strcpy / strncpy ===\n");
    char src[] = "C Language";
    char dst[32];
    strcpy(dst, src);
    printf("strcpy(dst, src) → %s\n", dst);

    char dst2[8];
    strncpy(dst2, src, sizeof(dst2));
    dst2[sizeof(dst2) - 1] = '\0';        // 널 종료 보장

    char str_buf[8]={0};                  // 모든 요소를 '\0'으로 초기화.
    strncpy(str_buf, src, sizeof(str_buf)-1);  // 마지막 한 칸을 널 문자를 위해 비워 둔다.
    printf("strncpy(str_buf, src, 7) → %s\n\n", str_buf);

    printf("=== 3. strcmp / strncmp ===\n");
    printf("strcmp(\"apple\", \"banana\") = %d\n", strcmp("apple", "banana"));
    printf("strcmp(\"same\", \"same\") = %d\n", strcmp("same", "same"));
    printf("strncmp(\"abcdef\", \"abcxyz\", 3) = %d\n\n",
          strncmp("abcdef", "abcxyz", 3));

    printf("=== 4. strcat / strncat ===\n");
    char buf[32] = "Hello";
    strcat(buf, " World");
    printf("strcat → %s\n", buf);

    strncat(buf, "!!!***", 3);  // "!!!"까지만 붙이기
    printf("strncat 3글자 → %s\n\n", buf);
```

```c
    printf("=== 5. strchr ===\n");
    char text[] = "Hello World";
    char* p = strchr(text, 'o');
    if (p != NULL)
    {
        printf("문자 'o'를 처음 찾은 위치부터: %s\n", p);
    }
    else
    {
        printf("문자 'o'를 찾지 못했습니다.\n");
    }
    printf("\n");

    printf("=== 6. strstr ===\n");
    char text2[] = "Game Development in C";
    char* q = strstr(text2, "Develop");
    if (q != NULL)
    {
        printf("\"Develop\" 를 찾은 위치부터: %s\n", q);
    }
    else
    {
        printf("\"Develop\" 를 찾지 못했습니다.\n");
    }
    printf("\n");

    printf("=== 7. strtok ===\n");
    char langs[] = "C,C++,Java,Lua,Unreal";
    char* token = strtok(langs, ",");

    while(token != NULL)
    {
        printf("토큰: %s\n", token);
        token = strtok(NULL, ",");
    }
```

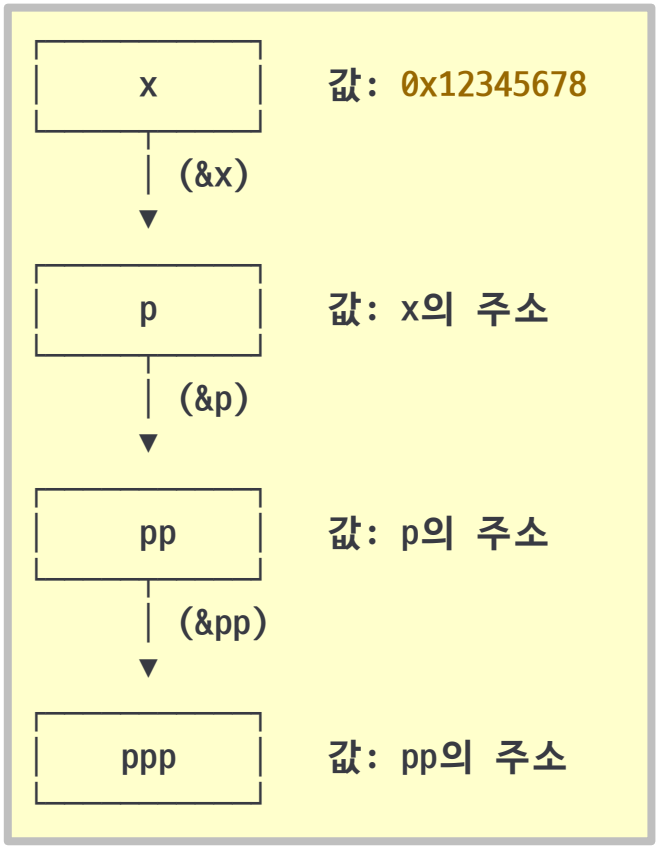
Heesung Oh
Copyright All rights reserved

포인터의 포인터

- 포인터의 주소를 저장하는 포인터

```
int x = 0x12345678;
int* p = &x;
int** pp = &p;
int*** ppp = &pp;

printf("%#x\n", ***ppp);  // 0x12345678
printf("%#x\n", **&*&*&*&*&*&p); // 0x12345678
```

```
ppp    → *ppp    → **ppp    → ***ppp
(ppp)    (pp)       (p)        (x)

***ppp == x == 0x12345678
```



9.6.2 이중 포인터 예제

Heesung Oh
Copyright All rights reserved

- C 언어의 함수 전달은 값 전달 (Call by Value)
- 포인터는 주소라는 값을 전달
- 포인터가 가리키는 주소를 변경하려면 → 이중 포인터 사용

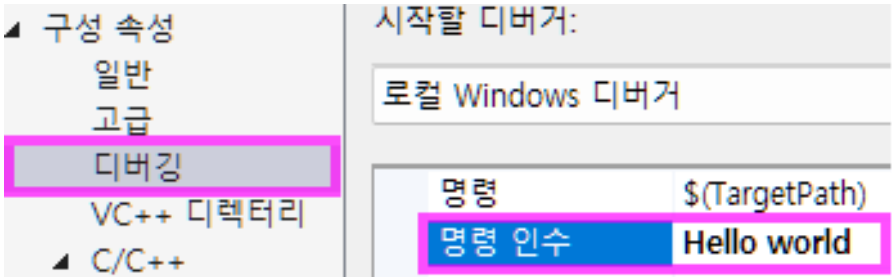
```c
void ChangePointer(int* p)
{
    static int a = 100;
    p = &a;     // p 값만 변경됨 (복사본)
}

int main(void)
{
    int x = 10;
    int* ptr = &x;

    // ptr 안의 주소가 '값 복사'로 전달됨
    ChangePointer(ptr);

    // 결과: 여전히 x(10)을 가리킴
    printf("%d\n", *ptr);
}
```

```c
void ChangePointer(int** pp)
{
    static int a = 100;
    *pp = &a; // 원본 포인터의 값 자체를 변경
}

int main(void)
{
    int x = 10;
    int* ptr = &x;

    // 포인터 변수의 주소를 전달함
    ChangePointer(&ptr);

    printf("%d\n", *ptr);  // 100
    return 0;
}
```

9.6.3 char* argv[] vs char**

Heesung Oh
Copyright All rights reserved

- char* argv[] → 포인터 배열
- char** argv → 포인터를 가리키는 포인터




```c
int main(int argc, char** argv)
{
    printf("인수 개수: %d\n", argc);

    for (int i = 0; i < argc; ++i)
        printf("argv[%d] = %s\n", i, argv[i]);
```

```
출력:
인수 개수: 3
argv[0] = <output folder>myPointer.exe
argv[1] = Hello
argv[2] = world
```

9.7 포인터를 함수 인자로 전달하기

Heesung Oh
Copyright All rights reserved

- 함수 호출
  - 값에 의한 호출 (Call by Value)
  - 포인터 전달 → 주소 값 복사
- 포인터 사용 구분
  - 싱글 포인터(int*) → 값 변경
  - 이중 포인터(int**) → 포인터 변경
  - 이중 포인터(void**) → 범용 포인터 처리

```
void swap(int* a, int* b) {
  int temp = *a;  // 임시 변수에 *a 값을 저장.
  *a = *b;        // b가 가리키는 주소의 값을 a 주소가 가르키는 주소에 저장.
  *b = temp;      // temp 값을 b 주소가 가르키는 주소에 저장.
}
```

```
void swap_ptr_int(int** a, int** b) {
  int* temp = *a; // a가 가리키는 포인터(즉, 원본 포인터)의 값을 temp에 보관.
  *a = *b;        // a가 가리키는 실제 포인터 값을 b가 가리키는 포인터 값으로 변경.
  *b = temp;    // b가 가리키는 포인터 값을 temp(원래 a가 가리 켰던 포인터)로 변경.
}
```

```
void swap_generic(void** a, void** b) {
  void* temp = *a;
  *a = *b;
  *b = temp;
}
```

9.8 const와 포인터의 조합

Heesung Oh
Copyright All rights reserved

- **포인터와 const**

선언 형태	포인터 변경	값 변경	설명
const int* p	O	X	읽기 전용 데이터에 대한 포인터
int* const p	X	O	포인터 고정, 데이터 수정 가능
const int* const p	X	X	포인터 고정, 값도 읽기 전용
const int (*p)[4]	O	X	4칸 int 배열을 읽기 전용으로 접근
int (*const p)[4]	X	O	4칸 배열을 가리키는 고정 포인터
const int* a[]	O	X	포인터 배열, 값은 읽기 전용

9.8.1 const 자료형 p (가리키는 값은 수정 불가)

Heesung Oh
Copyright All rights reserved

```
const int* p;
int const* p;    // 동일 표현
```

```
int x = 10;
int y = 20;
const int* p = &x;
*p = 30;      // 오류! 값 수정 불가
p = &y;       // OK 주소 변경 가능
```

9.8.2 자료형 const p (포인터 자체가 상수)

Heesung Oh
Copyright All rights reserved

```
int* const p = &x;
```

```
int x = 10, y = 20;
int* const p = &x;

p = &y;      // 오류! 포인터 변경 불가
*p = 30;     // OK 값 수정 가능
```

9.8.3 const 자료형 const p

Heesung Oh
Copyright All rights reserved

```
const int* const p = &x;
```

```
const int* const p = &x;
p = &y;    // X
*p = 20;   // X
```

9.8.4 배열 포인터와 const 결합

Heesung Oh
Copyright All rights reserved

```
const int* a[4];
int (*const p)[4];
```

```
int arr[][4] = { /* ... */ };

// 1) 배열 요소가 const → 요소 수정 불가
const int (*ptr1)[4] = arr;
ptr1[0][1] = 30;                // 오류: 요소가 const int

// 2) 포인터가 const → 포인터 이동 불가, 요소 수정 가능
int (*const ptr2)[4] = arr;
ptr2 = 다른주소값;               // 오류: 포인터가 const
ptr2[2][3] = 200;               // 가능: 요소는 int
```

9.8.5 함수 매개변수에서 const

Heesung Oh
Copyright All rights reserved

- 매개 변수의 const
  - 값 변경 시도 컴파일 오류
  - Read only 의도를 분명히 드러냄
  - 호출자는 매개변수가 수정되지 않는다는 점을 보장
  - 큰 배열을 값 복사 없이 효율적으로 전달

```
void PrintArray(const int* arr, int size)
{
  for (int i = 0; i < size; ++i)
    printf("%d ", arr[i]);
}
```

```
void PrintMessage(const char* msg)
{
  // msg가 문자열 리터럴이건 배열이건 상관 없이 안전하게 출력 가능
  printf("%s\n", msg);
}
```

학습 정리

Heesung Oh
Copyright All rights reserved

- **포인터의 본질**
  - 포인터 = 주소를 저장하는 변수
  - 자료형 → 메모리 해석 방식 결정
  - 캐스팅 → 주소 변경 X, 해석 방식 변경

- **포인터 연산과 배열 관계**
  - *, [] → 간접 접근
  - 포인터 연산 → 연속 메모리 순회
  - 배열 이름 = 상수 주소
  - 포인터 = 변경 가능한 주소
  - sizeof 차이 이해

- **배열·다중 포인터 구조**
  - 포인터 배열 vs 배열 포인터
  - 다차원 배열 ↔ 포인터 관계
  - 엔디안 → 바이트 단위 접근으로 확인

학습 정리

Heesung Oh
Copyright All rights reserved

- 문자열과 포인터
  - 문자열 = char 배열 + '\0'
  - 문자열 리터럴 → 읽기 전용
  - char [] vs char* 차이
  - 문자열 함수 → '\0'까지 순차 탐색

- 함수 인자와 const
  - Call by Value → 주소 복사
  - int* → 값 변경
  - int** → 포인터 변경
  - void** → 범용 처리
  - const → 값/주소 변경 여부 명확화

실습 및 과제

Heesung Oh
Copyright All rights reserved

- 문자열 뒤집기
  - 원본 배열 직접 수정 (ReverseString)
  - 결과를 별도 버퍼에 저장 (ReverseStringToBuffer)
  - 포인터 이동 + swap / 복사
- Trim (앞뒤 공백 제거)
  - 문자열의 시작/끝 공백 제거
  - 중간 공백 유지
  - 포인터로 범위 탐색 + 문자열 재정렬
- 가장 긴 단어 찾기
  - 공백 기준 단어 분리
  - 가장 긴 단어를 별도 버퍼에 저장
  - 포인터로 시작/끝 추적